INSOMNIA

Evaluate with confidence.
Decide with evidence.

The complete product & workflow guide

An account-based workspace that turns IOMETE buying and migration questions into requirements, sourced guidance, test plans and decision-ready evidence.

9	25	AI
server tool engines	workspace routes	local model assistance

Edition 2.0 / November 1, 2026

Current status: evaluation workflows, shared workspaces, recovery, OIDC SSO and local integrations have a working backend. Live IOMETE and Tavily connectors require credentials; public deployment requires a host.

Built for teams that want a clear, evidence-led IOMETE decision, with guidance, local AI, workload review and collaboration in one place.

Why Insomnia exists

PRODUCT PURPOSE / tools, everyday workflow, AI, integrations, collaboration and setup

A data platform can sound compelling while leaving the buying decision unresolved. Buyers still need to know which workloads fit, what the infrastructure needs, what the costs mean, and what evidence will justify a pilot or purchase. Insomnia organizes that work around one company evaluation.

The job it completes

Define a workload, document the buyer's success criteria, map needs to official product information, assign tests, import observations, and export a brief that separates documented facts from assumptions and unknowns.

Design strength	Practical value
Connected company context	Tools use the selected evaluation and save their outputs with its requirements and source links.
Traceable rules and calculations	Cost totals, prerequisite checks and benchmark gates are explicit rather than unexplained scores.
Evidence discipline	Unsupported security claims stay unknown. A capability description is not treated as customer assurance.
Report-level benchmark gates	Critical reports, correctness, concurrency and missing samples remain visible even when aggregate results look good.
Real persistence and accounts	Server-side validation, SQLite storage, authenticated sessions and account isolation support work across visits.
Reviewable handoff	Markdown briefs, saved tool snapshots, JSON backups and audit records make the evaluation easier to inspect.

The Insomnia advantage

Insomnia combines the buying journey with workload, deployment and migration review in one traceable workflow. Questions, source material, cost assumptions, test ownership and saved outputs stay connected. Teams can explain a decision, revisit its assumptions and hand it to the next stakeholder. This is a strong product proposition; a market-wide superiority ranking has not been independently established.

Who it serves

Buyers define success; sales engineers structure demonstrations and pilots; security leads identify assurance gaps. The perspective selector changes the interface context. It does not grant security permissions. Team access is separately enforced as owner, editor or viewer.

Understand, qualify, estimate

FEATURE GUIDE / every tool is executed and saved through the authenticated backend

01 Product guide

Why it matters: Give each stakeholder an understandable, source-linked starting point.
Inputs: A question and stakeholder role.
How it works: Matches the question to a curated official-documentation snapshot and selects role-specific follow-up questions.
Output: A source-linked explanation, relevant topics and the next questions to ask.
Scope: This is deterministic retrieval, not live web research or a connected language model. Unsupported topics are reported.

02 Use-case finder

Why it matters: Focus the pilot on the needs and constraints that matter to the buyer.
Inputs: Needs such as customer-controlled infrastructure, analytics, Oracle modernization, open tables and governance.
How it works: Applies visible, equal-weight rules; surfaces transactional-point-lookup concerns and a fully managed SaaS mismatch.
Output: A shortlist of evaluation use cases, constraints and a proposed next step.
Scope: The result is a screening aid, not a certification that a particular workload will fit.

03 Deployment readiness

Why it matters: Identify infrastructure gaps before committing to a deployment plan.
Inputs: Environment, Kubernetes, storage, PostgreSQL, ingress, CPU/RAM and an operating owner.
How it works: Compares supplied values with documented installation floors; marks missing values unknown and checks offline delivery for air-gapped setups.
Output: A prerequisite gap list with explicit confirmation and production-sizing questions.
Scope: It does not inspect a cluster. Version-specific requirements and platform exceptions require confirmation.

04 Business case

Why it matters: Make the cost argument explainable and expose its assumptions.
Inputs: USD license inputs, current cost, infrastructure, operations, services, migration, overlap and time horizon.
How it works: Calculates recurring and transition costs, applies the Enterprise list-estimate minimum or entered quote, and derives savings and simplified payback.
Output: A line-by-line, exportable cost case showing the supplied assumptions.
Scope: No taxes, inflation, discount rates, workload capacity or productivity gains are inferred. Pricing is a dated snapshot, not a quote.

How outputs stay reviewable

Each successful run captures inputs, result sections, source topics and a server timestamp. Results can be revisited, exported individually, or included in the company brief. Rerun after changing the evaluation or measurements; a saved output is a historical snapshot.

Prepare, test, decide

FEATURE GUIDE / no unexecuted test is presented as a measured success

05 Call preparation

Why it matters: Turn a meeting into a focused conversation with an agreed next decision.
Inputs: Evaluation context, stakeholder, duration, supplied company facts and the desired decision.
How it works: Builds a timed agenda, role-specific questions and a list of unresolved tests.
Output: A practical preparation brief tied to the buyer's evaluation.
Scope: The separate Integrations page provides local fictional profiles or configured Tavily research. Review cited pages and transfer relevant evidence into call preparation.

06 Demo planner

Why it matters: Demonstrate a relevant workload with clear checkpoints.
Inputs: Industry, primary concern and demo duration.
How it works: Selects an industry-specific synthetic schema, scenario and roles; adds focus-specific checkpoints and timed stages.
Output: A runbook for banking, manufacturing, retail or healthcare.
Scope: The runbook does not execute a demo or claim customer outcomes.

07 Security questionnaire

Why it matters: Reduce questionnaire effort while keeping unsupported claims visible.
Inputs: Questions entered as lines or imported from a one-column CSV.
How it works: Conservatively matches public capability topics; flags compound questions and unsupported assurance requests; calculates the unknown rate.
Output: Review drafts with sources, unknowns and reasons for further investigation.
Scope: It does not certify compliance or infer customer-specific encryption, recovery, contractual or audit assurances.

08 Benchmark review

Why it matters: Make acceptance depend on critical reports, correctness and concurrency.
Inputs: Report inventory, imported measurements, required concurrency and minimum sample count.
How it works: Computes nearest-rank p95 per report; checks sample coverage, concurrency and correctness; highlights critical report failures.
Output: A report-level pass, failure or insufficient-evidence assessment.
Scope: Imported measurements are user-provided. Integrations can run SELECT queries through a configured IOMETE driver; local sandbox timings are not cluster benchmarks. Query results are not automatically accepted as correctness evidence.

09 Migration review

Why it matters: Identify migration review work before committing to a SQL move.
Inputs: SQL, critical report targets, and CDC/reconciliation confirmations.
How it works: Scans SQL for selected Oracle-to-Spark review flags after removing comments and quoted literals; surfaces migration gates.
Output: A review checklist for syntax, correctness, concurrency, source permissions and cutover.
Scope: It is a heuristic reviewer, not a SQL compiler, automatic translator or performance predictor.

Use Insomnia, end to end

USER GUIDE / the new account starts empty; sample data is optional

1. Create an account or sign in

Use the Insomnia server account form. Passwords must be 12-128 characters. No IOMETE account or external API key is required for the current toolset.

2. Create an evaluation

Enter the company, workload name, platform, owner, target date and success criteria. The backend creates linked proposed acceptance tests with unknown results.

3. Build the workload inventory

Add reports manually or import a CSV. Baselines and targets are buyer-provided seconds. Identify business-critical reports and agree the concurrency target.

4. Run the relevant tools

Select the company evaluation and supply its facts. Run guidance, fit, deployment, cost, call, demo or security tools. Inputs and outputs are saved after server acknowledgment.

5. Record and review evidence

Assign test owners and criteria. Import actual measurements for existing report names, run benchmark review, and record observations. The app does not silently mark acceptance tests passed.

6. Export the decision brief

Download company plans or tool reports as Markdown. Inspect saved snapshots, server audit and deadline blockers. Use a JSON backup before replacing or clearing data.

CSV headers

```
Workload inventory:
report,baseline,target,critical

Benchmark observations:
report,seconds,concurrency,correct

Security questionnaire:
question
```

CSV values must meet the relevant validation rules. Use true/false for critical and correct fields; quote cells containing commas. Templates can be downloaded from the matching screen. No example measurement is inserted as a real result.

A working backend

TECHNICAL GUIDE / Node.js + SQLite / one origin for UI and API

Browser form -> authenticated JSON API -> validation and tool engine -> SQLite transaction -> new revision and audit event -> confirmed UI state. The frontend does not mark a write saved or clear a submitted form before server confirmation.

API area	Responsibility
GET /api/health	Reports available engines and integration capability status.
POST /api/auth/signup, /login, /logout	Creates accounts, verifies passwords, establishes sessions and revokes logout sessions.
GET /api/session, /workspace	Resolves the signed-in account and reads its saved workspace and revision.
PUT /api/workspace	Validates evaluation, requirement, workload, test, settings and deletion changes; enforces revisions and immutable tool outputs.
POST /api/tools/run	Executes all nine tool engines against saved account data and persists authoritative results.
POST /api/import	Parses workload, measurement and security-question CSV on the server.
POST /api/workspace/restore, /reset	Validates backups, marks imported snapshots and handles explicit sample or empty resets.
GET /api/export	Creates company briefs, saved report downloads and workspace JSON backups from persisted records.
GET /api/insights, /audit	Computes dashboard metrics and reads selected-workspace audit history.
POST /api/auth/forgot, /reset	Delivers a one-time 30-minute link and revokes sessions on reset.
/api/auth/sso/*; /api/team/*	Signed OIDC code flow with PKCE, workspace invitations, switching and permissions.
/api/integrations/*; /api/operations	SQL, research and assistant runs with provenance; operational readiness checks.

Data integrity and concurrency

Each account owns a personal workspace and can join shared workspaces through email-bound invitations. Saves compare the client revision with the current database revision, then commit data and an audit entry in a transaction. Stale revisions are rejected, the latest data is reloaded, and the user can review and retry. Ordinary client edits cannot create or rewrite generated tool results.

Account and session controls

Accounts use salted scrypt password hashes, opaque HTTP-only session cookies, SameSite=Strict, CSRF and origin checks, and authentication rate limiting. Logout revokes the server session. The database persists across server restarts and is excluded from Git.

Run, save, recover

OPERATIONS / current implementation is a single-server application

Development

```
Node.js 24.11 or later
npm ci
npm run dev
```

Open the printed URL, normally http://127.0.0.1:5181/. Development selects a free port when needed, so always use the exact URL printed in the terminal. The account screen is /portal/#auth. Run the full-stack server to keep the UI and API connected.

Production serving

```
npm run build
npm start
```

The production server serves the built frontend and the same authenticated API. The default listener is local-only. Remote hosting needs HTTPS, secure cookies, a configured public origin, a durable database volume and deployment operations.

Concern	Current behavior
Persistent storage	.data/insomnia.sqlite. Database and environment files are private and ignored by Git.
Backup and restore	JSON contains workspace records, not account passwords or sessions. Restored report snapshots are explicitly marked imported.
Previous browser data	Use Settings to import previous browser data. For an old app on another port, export its JSON there and restore it into the server account.
Conflict or connection failure	Inputs remain available. Retry connection or reload the latest revision before deliberately resubmitting a change.
Limits	Requests: 5 MB. Saved workspace: 4 MB. CSV and row limits are enforced. Snapshots retain the latest 2,000 events; database audit is separate.
Account scope	Personal and invited workspaces are separate. Owners manage access, editors save changes, and viewers read. The perspective selector grants no access.

Configuration is described in .env.example and BACKEND.md. A workspace JSON export is not a full database backup. Use a suitable SQLite backup procedure or stop the server before copying database files. Node's bundled SQLite API currently emits an experimental-feature notice.

What is done. What is next.

IMPLEMENTATION STATUS / assessed against the current repository

Status	Scope
Implemented and tested	All nine tool engines execute through authenticated backend endpoints.
Implemented and tested	Account creation, password sign-in, session logout, per-account storage, revisions and server validation.
Implemented and tested	CSV imports, persisted outputs, exports, settings, custom tests, metadata edits, resets, backups and server audit.
Implemented and tested	Dashboard counts, weekly activity, completion and deadline blockers use persisted records and server calculations.
Browser verification	Registration, session recovery, SQL calculations, local profiles and assistant flows reviewed in the browser. Independent accessibility and broad device coverage remain separate.
Implemented; credentials required	Official IOMETE SQLAlchemy connector, bounded SELECT execution and persisted query results. No live cluster is available. Local SQL runs on synthetic SQLite data.
Local and configurable	Ollama model inference, sourced retrieval, fictional company profiles and a Tavily connector. Live Tavily research requires a key; documentation refresh is manual.
Implemented and tested	Shared workspaces, email-bound invitations, owner/editor/viewer permissions, OIDC SSO and one-time password recovery. Local mail uses private outbox files; SMTP is configurable. Signup email verification is separate.
Required before broad production use	Deployment monitoring, restore drills, load testing, independent security review, operational ownership and multi-server strategy if needed.

Verification evidence

Local verification covers all nine tool engines, account isolation, CSRF, persistence after restart, stale revision rejection, CSV validation, exports and logout. Browser review has also created an evaluation and report, saved all nine tool outputs, imported questionnaire and benchmark CSVs, calculated p95 and downloaded a report. Server-render checks exercise 25 routes with populated and empty workspaces. Production-mode smoke checks verify built assets, API guards, private files, restart persistence and database backup/restore. These checks do not replace an independent security or load review.

Is the backend done?

The backend is implemented for the current evaluation-workspace scope. It is not accurate to call every possible integration or production concern complete. Local service workflows are available without paid providers. External verification still needs an IOMETE cluster, research credentials, a real mail domain and a public deployment target. A local fixture is never presented as live external evidence.

Local first. Live when configured.

SERVICE SETUP / no paid account needed for local workflows

Language model

Install Ollama and run ollama pull qwen2.5:1.5b. Set OLLAMA_MODEL=qwen2.5:1.5b in the ignored .env. The local model creates review drafts with supplied evidence. Without a model, the assistant uses labeled local retrieval. A cold start can take up to a minute.

SQL execution

Development uses a bounded, isolated SQLite sandbox with synthetic sales and customers tables. For live IOMETE, install requirements-iomete.txt and set IOMETE_SQLALCHEMY_URL from your compute connection details. Use a SELECT-only principal. No query timing is silently imported as verified benchmark evidence.

Company research

Local profiles cover Northstar Bank, Meridian Manufacturing, Harbor Analytics and Cedar Insurance. They are fictional practice data. Set TAVILY_API_KEY for real web search; the app retains links and excerpts rather than inventing company facts.

Recovery and invitations

Development emails are private JSON messages under .data/mail; they are not sent externally. Configure SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD and SMTP_FROM for real delivery. Reset tokens are hashed, expire in 30 minutes, work once and revoke existing sessions. Invitations are email-bound and expire in 48 hours.

Single sign-on

Development starts a loopback OIDC provider. Generated credentials are in .data/local-sso.json. For an enterprise provider, configure OIDC_ISSUER, OIDC_CLIENT_ID and OIDC_CLIENT_SECRET. Register /api/auth/sso/callback at the public origin. Existing accounts must sign in first before linking SSO; matching email alone cannot take over an account.

Deployment and recovery

Run npm run check:production for an isolated production-mode and restore drill. Run npm run backup for a verified SQLite backup. Docker and Compose definitions are included. Public rollout still requires a durable host, HTTPS, secure cookies, external service configuration and operational ownership. Disable local services and the local identity provider in production.

Ollama: practical AI assistance

LOCAL AI / qwen2.5:1.5b / source context stays visible

Insomnia can run a real local language model through Ollama. The configured model in the local setup is qwen2.5:1.5b. This brings a question-and-answer workflow into the same workspace as the evaluation, reports and source material.

Why it is necessary

Teams need to understand platform concepts and turn technical information into useful conversations. A contextual assistant helps them begin that work while keeping the evaluation and the evidence behind the answer close at hand.

How it works

1. Enter a question in Integrations and optionally select an evaluation.
2. Insomnia retrieves relevant passages from its curated documentation snapshot.
3. The server sends the question, evidence and selected buyer context to the configured Ollama endpoint.
4. The model returns a short draft. The UI shows the draft and retrieved evidence together.
5. A successful run is saved in the selected workspace with its execution mode and timestamp.

Why this approach is useful

Users can compare an answer with the supplied evidence, keep a useful explanation for the next conversation, and see whether it came from a model or local retrieval. The default Ollama address is on the local computer; another configured endpoint receives the context sent to it.

Enable the local model

```
ollama pull qwen2.5:1.5b
OLLAMA_URL=http://127.0.0.1:11434
OLLAMA_MODEL=qwen2.5:1.5b
```

Two useful modes

With a model configured, the assistant performs real inference. Without a model, it returns matching local documentation and labels that mode local retrieval. The deterministic Product guide remains available separately.

A clear responsibility boundary

Generated text is a review draft. The current integration limits generation to 160 tokens and asks for concise answers. A cold model may take up to a minute. The model has no tools: it cannot execute SQL, change evaluation data, certify compliance or verify performance. Review generated claims against the displayed evidence.

SQL and research with visible sources

INTEGRATIONS / local practice and configured live services

SQL execution: why it matters

A team needs to move from a plan to a concrete query result. Insomnia provides a place to run a bounded read-only query and preserve the returned rows, timing and execution mode with the workspace.

Mode	How it works and what the result means
Local SQL sandbox	An isolated SQLite worker executes SELECT queries on fictional sales and customers tables. Results demonstrate the workflow; timing is not IOMETE performance.
Live IOMETE connector	The official IOMETE SQLAlchemy driver uses an operator-configured connection. Supply the cluster connection and a SELECT-only principal. No live cluster has been supplied for external verification.

Example local query

```
SELECT region, SUM(revenue) AS revenue
FROM sales GROUP BY region
```

Synthetic sales total 4,000: North 1,800, South 800 and West 1,400. The local worker has a two-second deadline. Live query processes have a 20-second client deadline. Results are limited to 100 rows. Check the cluster after a live timeout; stopping the client does not establish remote cancellation.

Company research: why it matters

Good call preparation starts with relevant facts and a source another person can review. Insomnia keeps research results separate from assumptions so the team can choose what to carry into its evaluation.

Local profiles and live search

Local mode includes fictional Northstar Bank, Meridian Manufacturing, Harbor Analytics and Cedar Insurance scenarios. Unknown companies stay unknown. With TAVILY_API_KEY configured, Insomnia sends the company query to Tavily and stores up to five public source links and excerpts. The user reviews those pages before relying on a claim.

Saved integration history

SQL, research and assistant results retain their inputs, mode and time in the selected workspace. Owners and editors can run integrations; viewers can read saved results. Provider failures produce an error rather than an invented successful answer.

Work together, keep ownership clear

TEAMWORK / shared workspaces, identity and recovery

Insomnia connects the people who define success, prepare the pilot and review the evidence. Every account keeps a personal workspace and can join other workspaces through an email-bound invitation.

Access	What the person can do
Owner	Read and change workspace data; run integrations; invite members; change access; remove members and revoke invitations.
Editor	Read and update evaluation work and run integrations. Access management stays with the owner.
Viewer	Read and export evaluation work and read saved integration results. The server blocks changes and integration execution.

Why shared workspaces are useful

The evaluation, workload inventory and saved outputs stay connected as responsibilities move between people. Team shows membership, pending invitations and access history. Revision checks reject stale overwrites instead of silently accepting them.

Invitation and workspace flow

The owner enters a teammate email and access level. The invitation expires after 48 hours and must be accepted by the matching signed-in account. Users can switch between personal and invited workspaces. Membership and permissions are checked on each request; removing a member removes shared access from active sessions.

Single sign-on

OIDC SSO uses an authorization code flow with PKCE, state, nonce and signed ID tokens. Local development includes a loopback provider for a real test flow. Enterprise setup uses OIDC_ISSUER, OIDC_CLIENT_ID and OIDC_CLIENT_SECRET with the exact /api/auth/sso/callback URL. Existing password accounts sign in before linking SSO; email matching alone cannot take over an account.

Recovery and delivery

Forgot password requests a one-time, hashed reset token that expires in 30 minutes. Resetting the password revokes existing sessions. Development messages are private files under .data/mail. SMTP settings enable external delivery; mailbox delivery needs a real sender configuration. The local outbox is not Internet email.

Perspective and permission

Buyer, Sales engineer and Security lead describe the evaluation perspective. Owner, Editor and Viewer control access. Keeping these separate makes responsibilities understandable and permissions enforceable.

Everyday features, one decision

WORKSPACE REFERENCE / what each feature does and why it matters

Feature	How it works and why it matters
Evaluation overview	Counts, weekly activity, completion and blockers derive from persisted records. See what has been recorded and what needs an owner or result.
Evaluations and companies	Create, edit, search and remove evaluation records. Company views connect related pilots and requirements for an easier handoff.
Workloads and requirements	Record report baselines, targets, criticality and buyer success criteria. Give the pilot a concrete question to answer.
Test plans	Add acceptance tests, assign owners, set criteria and record observations. A pass or failure requires a recorded result.
Evidence library	Open curated official sources and distinguish documented capabilities, buyer inputs, proposed tests and unknowns.
Saved reports	Revisit input/output snapshots, filter them, export individual reports or all reports, and remove snapshots no longer needed.
Reports and exports	Download evaluation briefs and tool outputs as Markdown so stakeholders review the same assumptions and evidence.
CSV imports and templates	Import workloads, measured results and questionnaire rows with validated headers and values. Reuse existing evidence without rebuilding each row.
Activity and audit	Inspect server-recorded changes, dates and revisions. Keep the evaluation history visible through edits and imports.
Blockers and notifications	Surface unfinished tests, missing ownership and target-date follow-up. Put the next evidence gap where users can find it.
Settings and recovery	Set workspace name and reminder window, load optional examples, start empty, migrate prior browser data and restore validated JSON backups.
Full database backup	Create an integrity-checked SQLite copy. A workspace JSON export contains domain records, not all accounts, team records or integration history.
Responsive navigation	Navigation folds into a mobile sheet. Search, source links and exports keep the same purpose across screen sizes.

The practical advantage

A question can become a documented requirement, a planned test, a measured observation and a shareable decision brief. AI and integrations add assistance while calculations, inputs and evidence remain inspectable.

Sources and next steps

REFERENCE / official product facts are separate from Insomnia implementation claims

IOMETE source snapshot reviewed November 1, 2026. Features, pricing, entitlements and requirements may change; validate current terms and the applicable product version before a customer decision.

IOMETE deployment options

https://iomete.com/product/deployment

On-premises installation requirements

https://iomete.com/resources/deployment/on-prem/install

Technical architecture

https://iomete.com/resources/blog/iomete-core-components-technical-architecture

Oracle-to-Spark migration experience

https://iomete.com/resources/blog/oracle-to-spark-migration

Pricing and commercial plan details

https://iomete.com/pricing

Software license and operating responsibility

https://iomete.com/policies/software-license-agreement

Ollama chat API

https://docs.ollama.com/api/chat

IOMETE SQLAlchemy driver

https://iomete.com/resources/user-guide/driver/iomete-sqlalchemy

Tavily search API

https://docs.tavily.com/documentation/api-reference/endpoint/search

Insomnia source repository

https://github.com/harsh-dixit21/Insomnia

Repository documentation

README.md covers setup and the feature guide. BACKEND.md documents endpoints and server behavior. tests/ contains the algorithm and HTTP integration checks; scripts/check-screens.mjs performs component render checks.